

Predicting Counter-Strike tactics using Graph Neural Network based Machine Learning

SZABOLCS CSAPÓ, University of Twente, The Netherlands

ABSTRACT

Counter-Strike has been one of the largest esports scenes for over a decade, with gameplay increasingly defined by complex team tactics. As tactics have evolved, distinguishing and analyzing them has become more challenging. This research automates tactic recognition in each round using a Graph Neural Network (GNN). The developed GNN model achieved an accuracy of 81.17% and an F1-score of 0.6945, demonstrating strong performance in identifying both common and ambiguous tactics. Results show that a simple, two-layered Graph Convolutional Network, combined with all available node features (health, armor, player position, and utility), provides the most effective way of predicting Counter-Strike tactics. The model's ability to generate reliable predictions from partial in-game data offers significant potential for improving coaching strategies and automating data labeling. These findings advance the state of automated tactic recognition in esports and contribute to the broader development of the esports scene.

Additional Key Words and Phrases: Counter-Strike, Graph Neural Network, Machine Learning

1 INTRODUCTION

Counter-Strike is a widely recognized first-person shooter game where two teams, Terrorists and Counter-Terrorists, compete for objectives such as planting or defusing a bomb or eliminating the opposing team. Known for its fast-paced, team-based gameplay and strategic depth, Counter-Strike has emerged as one of the most influential titles worldwide in the rapidly growing esports industry, which is projected to reach US\$5.9 billion by 2029 [1]. The game's popularity is driven by its emphasis on coordination, real-time decision-making, and the execution of complex tactics, which are essential for competitive success.

Despite the increasing availability of in-game data and the rise of data-driven analytics in traditional sports, the field of automated tactic recognition in Counter-Strike remains underdeveloped. In traditional sports, data analytics has seen significant advances, such as *TacticAI* for football, which leverages graph neural networks to analyze and recommend corner kick strategies [2], and *PIVOT* for handball, which employs deep learning to value player actions based on tracking data [3]. Similarly, other major esports titles—such as *League of Legends*, *Overwatch*, and *Valorant*—have increasingly adopted data-driven approaches for tactical analysis. In *League of Legends*, automated systems track lane rotations, jungle paths, and team fights to provide coaches and players with actionable insights for optimizing team efficiency. Platforms like *Mobalytics* offer performance metrics and personalized recommendations, supporting

data-driven decision-making both before and during matches. Likewise, *Valorant* features interactive analytic tools that guide players on how to maximize the effectiveness of utility usage, such as smoke and flash grenades.

However, there has been relatively little research into automated, data-driven tactical analysis for Counter-Strike. Most existing work relies on manual review and subjective interpretation, which is both time-consuming and prone to inconsistency [6]. This knowledge gap limits the ability of teams and analysts to derive actionable insights from the richness of available data, potentially hindering performance improvement and strategic adaptation. This research addresses the current knowledge gap by investigating the accuracy to which graph-based machine learning models can automate the recognition and prediction of tactics in Counter-Strike. By developing and evaluating a model that processes in-game data and predicts team-level tactics, this work aims to contribute to the broader development of automated coaching tools and data-driven decision-making in esports.

2 PROBLEM STATEMENT

Given the nature of the game and the way the servers already receive large amounts of data each frame, there is a large amount of data collected in esports. These datasets include the absolute position of the players, their positions related to bombsites, the position of the bomb, the time left in the round, the money spent by each team in the round, and many more valuable assets to the game.

However, there has not been enough research on automated data analysis models in Counter-Strike. Not using the available data to its fullest can hinder team performance by not implementing data-driven decision-making in their game [4].

2.1 Research Question

To address the aforementioned problem, this research focuses on the following research question:

- "To what accuracy can Graph Neural Networks predict pre-defined tactics (e.g., site execution, fakes) in Counter-Strike from partial round data?"

3 BACKGROUND

The increasing availability of in-game data in esports has opened up new possibilities for data analytics with machine learning. Counter-Strike is a prime example, with large public datasets enabling detailed gameplay analysis. However, the development of automated data analysis tools has not kept up with the pace.

3.1 Counter-Strike tactics

Counter-Strike is a team-based first-person shooter (FPS) set in a three-dimensional environment, where two teams of 5 players (Terrorists and Counter-Terrorists) compete across a variety of maps.

TS&IT 43, July 4, 2025, Enschede, The Netherlands

© 2025 University of Twente, Faculty of Electrical Engineering, Mathematics and Computer Science.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee.

Each match is divided into rounds, with teams switching sides after the first half. There are several ways a team can win a round.

- Terrorists
 - Eliminate the Counter-Terrorists
 - Plant the bomb on a bombsite and guard it until it explodes
- Counter-Terrorists
 - Eliminate the Terrorists
 - Prevent the Terrorists from planting the bomb until the time runs out
 - Defuse the bomb

Each map in Counter-Strike features a unique layout that encourages distinct tactical approaches. This research specifically targets tactic prediction on the Dust 2 map (see Figure 1), which is among the oldest and most established map designs in the game. Known for its balanced structure and enduring popularity in professional play, Dust 2 offers a robust foundation for analyzing and predicting team tactics.



Fig. 1. Overview of the Dust 2 map in Counter-Strike, highlighting bomb-sites A and B, T spawn, and CT spawn

A tactic in Counter-Strike can be defined as a coordinated sequence of actions undertaken by a team to achieve a specific, short-term objective within a round. Teams often execute tactics in order to increase their chances of winning a round. These tactics include:

- Rushing to a bombsite, where teams aim to reach a bombsite and plant the bomb as soon as possible.
 - This overwhelms the defending team, forcing them to act fast under pressure, increasing the likelihood of mistakes being made.
- Default setup, where teams aim to gather information without committing to a bombsite early on.

- By spreading out and securing key areas, the attacking team can gather information about the defending team’s weaknesses and exploit those.
- Faking a bombsite, where teams pretend to go to a certain bombsite, distracting the other team, while actually going to a different part of the map.
 - This draws the attention and resources of the defending team to a certain part of the map, away from the true target, creating opportunities elsewhere.

These tactical maneuvers require precise timing, communication, and the effective use of utility items like grenades and flashbangs. Understanding and analyzing these tactics is central to coaching and performance analysis, as it enables teams to identify patterns, exploit weaknesses, and optimize their approach to each map and opponent.

While human analysts’ work has been highly influential on match analyses of professional play, it is not humanly possible to process datasets such as the ESTA dataset [5]. The ESTA dataset contains 1500 professional matches played in online or LAN tournaments, capturing detailed player and location information at up to two frames per second. Each frame includes key attributes such as Position, Health, Armor, Equipment, Utility, providing a rich foundation for data analysis. The use of such a vast dataset opens up possibilities of automated machine learning approaches that are able to handle complex gameplay data.

Unlike in traditional sports, in Counter-Strike there is a lack of research on this topic. This research aims to close this gap by developing a GNN for Counter-Strike tactic prediction, improving data analytics in esports.

3.2 Data analysis in esports

The esports industry has grown rapidly, with professional teams and organizations increasingly relying on data-driven insights to optimize performance and gain a competitive edge. Game analytics in esports involves the collection, analysis, and interpretation of data related to gameplay, player behavior, and team performance. This includes descriptive analytics (what happened), diagnostic analytics (why it happened), predictive analytics (what will happen), and prescriptive analytics (what actions to take).

Several major esports titles, including League of Legends, Overwatch, and Valorant, have already implemented data-driven approaches for recognizing and analyzing in-game tactics [4]. In League of Legends, automated systems track lane rotations, jungle paths, and team fights to enhance team efficiency. Platforms such as Mobalytics provide performance ratings for players based on in-game data, such as gold earned, kill participation, and damage dealt. These systems also recommend optimal champion selections based on real-time data, offering teams advantages before the match begins. Similarly, in Valorant, analytics tools offer tutorials on maximizing the effectiveness of smoke and flash grenades. Despite differences in game design, the automation of data analysis and the adoption of data-driven strategies are growing trends across all major esports.

3.3 GNNs for tactic prediction

The use of Graph Neural Networks (GNNs) in sports analytics has become a highly researched topic in recent years. One of the first articles to propose this approach was TacticAI by Wang et al. [2], which used GNNs to transform and automatically analyze large amounts of data in football matches. This innovative design was later cited by several other researchers, who advanced data analytics by applying various GNN architectures.

In particular, Xenopoulos et al. [6] employed both Graph Convolutional Networks (GCNs) and Graph Attention Networks (GATs) to model complex interactions in American football and Counter-Strike: Global Offensive (CS:GO). GCNs operate by aggregating features from neighboring nodes using fixed weights, making them particularly effective for capturing static spatial relationships, such as player proximity on a football field. In contrast, GATs utilize dynamic attention mechanisms, enabling the model to assign varying importance to different interactions, for example, emphasizing a quarterback's connection to receivers in American football.

This approach demonstrated an improvement in prediction accuracy of approximately 20% over human analysts for American football. The superior performance of GNNs stems from their ability to capture both local and global tactical patterns by representing players as nodes and their interactions as edges within a graph. Moreover, the application of GNNs to Counter-Strike opened up new possibilities for data analysis by enabling interactive "what-if" scenarios. Through a user interface, analysts could dynamically modify player positions and equipment, and the GNN would recalculate potential outcomes in real time.

4 METHODOLOGY

This research consists of three essential parts of development: data preprocessing, building a GNN capable of predicting Counter-Strike tactics, and finally, the evaluation of the GNN model.

4.1 Data preprocessing

The main dataset used for training and testing the model is the ESTA dataset [5], which provides detailed round data from CS:GO matches. Each in-game round is a separate JSON file with data on money spent by a certain team, money saved from the last round, and the value of their equipment. In each frame of the round, data is collected about the player's positions, the bomb's position, if the bomb is planted or not, and the time remaining.

Graphs provide a natural and effective way to map Counter-Strike gameplay data, since they revolve around entities and their relations [4]. Each round of a Counter-Strike match is represented as a graph $G = (V, E)$, where V is the set of nodes (players, bombsites, and the bomb), and E is the set of edges capturing relationships such as distances between nodes. Each node $v_i \in V$ is associated with a feature vector, $x_i \in \mathbb{R}^F$ and the graph has a graph-level feature vector $g \in \mathbb{R}^Q$, which includes attributes such as time remaining and the tactic label. This graph-based representation is a natural and effective way to model Counter-Strike gameplay data, as it preserves the relational structure of player interactions and map elements.

Selecting the most effective combination of features is crucial for achieving optimal model performance in graph-based machine

learning tasks. In this research, the choice of node features was guided by both the considerations involved during manual labeling and empirical evaluation of their impact on prediction accuracy. The features included in the model, such as player position, health, armor, and utility usage, were evaluated systematically to determine which combinations provided the greatest predictive power and which features, when included, contributed primarily to noise or reduced model precision.

During the labeling process, annotators considered player positions, health, armor, and utility usage as key indicators of tactical intent. These features were then incorporated into the graph representation to ensure that the model could leverage the most relevant information available. During testing it was revealed that not all features contributed equally to model performance. For instance, the inclusion of health and armor alongside position features resulted in a slight decrease in precision, suggesting that these attributes may introduce noise or redundancy in certain contexts. In contrast, adding utility usage significantly improved both recall and precision, highlighting its importance as a strong signal for tactical classification.

Node features were chosen based on insights from the labeling process and their demonstrated predictive value, as indicated by analyses of the ESTA dataset. The final set of node features includes:

- Position (X, Y coordinates): Normalized using linear normalization to ensure consistent scaling across rounds on the Dust 2 map. Normalization is essential for machine learning models, as features on different scales can distort learning and result in biased outcomes.
- Health and Armor: Represented as integers, these features provide information about each player's survivability.
- Utility (number of grenades): Counted as integers, utility usage is a strong indicator of tactical intent and was found to significantly improve model performance during testing.
- Area ID and Node Type: Encoded using one-hot encoding. Area ID indicates the map region, while Node Type distinguishes between players, bombsites, and the bomb. One-hot encoding is a standard technique for categorical data in machine learning, as it avoids implying ordinal relationships and enables models to learn from distinct categories [7].
- isAlive and hasBomb: Binary features (1 for true, 0 for false), indicating whether a player is alive and whether they are carrying the bomb.

Graph-level features include:

- Time remaining in the round: Provides temporal context for the tactic.
- Tactic label: The ground truth for model training and evaluation.

The model is currently using linear normalization on the position values of the nodes. This is a highly effective normalization method if the nodes are in the same coordinate system, and since the model only analyzes Dust 2 games, it works well. However, if more maps are introduced to the model, this will not work, since every map has a unique layout, and an A setup could be positionally very different on Dust 2 and another map.

Missing or incomplete data can negatively impact model performance. In this work, only rounds with complete feature sets were included in the dataset, as the ESTA dataset is curated to include only complete data [5].

Graphs for each round are serialized and stored as .pkl files. The choice of pickle format was driven by its efficiency in storing complex Python objects, ease of use, and widespread adoption in Python-based machine learning libraries. Each round's graph is stored in a dedicated directory, making it easy to navigate through thousands of graphs and find the round or game currently needed.

```
project/
├── research_project/
│   ├── graphs/
│   │   ├── 00e7fec9-cee0-430f-80f4-6b50443ceacd/
│   │   └── 1a8afefa-b15e-43cf-adb9-83446d956aaf/
│   │       └── graph-rounds-1.pkl
│   ├── models/
│   ├── src/
│   └── tactic_labels/
```

4.2 GNN

Graph Convolutional Network (GCN) and Graph Attention Network (GAT) are the structures that were taken into consideration for the GNN model because of their demonstrated effectiveness in modeling spatial relationships and capturing dynamic relationships between players [7] [6]. GCNs perform best at modeling spatial structures by aggregating node features between neighboring nodes, while GATs introduce an attention mechanism that allows the model to focus on more relevant features. After evaluating both, the GNN architecture chosen for this model was a Graph Convolutional Network (GCN), as it was the one that showed the most accurate predictions during the testing phase. The structure is as follows:

- (1) A graph is built for each round using the structure described in 4.1.
- (2) The nodes update their features by aggregating neighbor information using message passing with a fixed weight over the edges [7].
- (3) Global Sum Pooling is applied to the embeddings of the graph in order to aggregate the node-level features into graph-level representation [7].
- (4) The model is trained with labeled data and tested on the test set from the ESTA [5] dataset.

The model was built using the PyG library in Python, specifically with the torch-geometric 2.6.1 version. Various numbers of hidden channels were tested, including 64, 128, and 248. The configuration with 128 hidden channels delivered the best performance: 64 channels resulted in slight underfitting during training, while 248 channels led to signs of overfitting.

During training, the AdamW optimizer was used with a learning rate of 0.0002 and a weight decay of 1e-5. AdamW is widely adopted in machine learning, particularly for neural networks and graph-based models, as it helps prevent overfitting by decoupling weight decay and adapts learning rates for each parameter, leading to faster

and more stable convergence [9]. Additionally, the StepLR scheduler was employed to improve learning, with a step size of 10 and gamma set to 0.5. This scheduler reduces the learning rate by a fixed factor every specified number of epochs, where an epoch is defined as one complete pass through the entire training dataset by the learning algorithm.

To address class imbalance in the dataset, class weights were computed and incorporated into the cross-entropy loss function. This approach ensures that underrepresented classes are not ignored during training, as the loss function penalizes misclassifications of rare classes more heavily. Using class weights is a standard technique for improving model performance on imbalanced datasets, as it helps the model learn to recognize all classes more effectively and prevents bias toward majority classes [7].

The dataset is split into training and test sets in an 80/20 split, where there is 80% training data and 20% test data, to ensure the best outcome of the model. With this approach, the model can learn complex patterns during training while still having enough data to validate its performance during the test phase. The 80/20 ratio is widely adopted in machine learning research as it helps prevent overfitting and ensures that the metrics achieved reflect the model's ability to predict unseen data.

4.3 Evaluation

To evaluate the model, accuracy, precision, recall, and F1 score are calculated.

- Accuracy measures the overall correctness of the model. It provides a baseline understanding of performance.
- Precision measures prediction reliability. It is essential for avoiding a large number of false positives.
- Recall measures detection capability. It ensures that the model does not miss subtle tactics.
- F1-score balances precision and recall. It is essential for imbalanced datasets, where both false positives and false negatives must be minimized.

In this research, the F1-score is the most important measure of the model's performance. It is the metric showing how the model can predict tactics in a balanced way, not skipping any tactics, but also correctly identifying the other ones.

5 RESULTS

The following section will explore the results achieved by the model during training and in predicting actual Counter-Strike matches.

5.1 Model performance

From Table 1 we can see that keeping all of the aforementioned features produced the best results in all of the metrics measured. Some interesting observations can also be made based on these results. When we introduce Health + Armor next to the Position values, we can see the Precision dropping by a substantial amount. This indicates that the added features are increasing the False Positive rate of the model. However, once the Utility is added instead, it increases both Recall and Precision, indicating an increase in True Positives and a decrease in False Positives. Adding additional features like Health and Armor increased these metrics even further.

Table 1. Feature combinations

Features	Accuracy	Recall	Precision	F1-score
Position	79.16%	0.7068	0.6301	0.6568
Position + Health + Armor	79.07%	0.7069	0.6130	0.6417
Position + Utility	80.09%	0.7131	0.6517	0.6707
All Features	81.17%	0.7510	0.6643	0.6945

Table 2. GNN architecture combinations

Model	Training Accuracy	Test Accuracy	F1-score
2-layered GAT	78.04%	78.10%	0.6831
2-layered GCN	82.79%	81.17%	0.6945
3-layered GAT	77.65%	77.16%	0.6692
3-layered GCN	81.94%	78.78%	0.6672

The 2-layered GCN had the highest accuracy and F1-score as well (see Table 2), meaning it was the most efficient way of predicting Counter-Strike tactics. The attention mechanism of the GAT architecture did not add any performance to the model. Since the graphs used have less than 10 nodes, the model favored a simple GNN architecture, with only 2 layers. Having more layers can lead to overfitting (which we did not see in this case) and also more complex tasks, as it exponentially increases computational power.

In conclusion, The 2-layered GCN architecture with all of the node features included was the most efficient way of training the model for Counter-Strike tactics prediction.

5.2 Confusion Matrix

Figure 2 shows the Confusion Matrix of the predictions of the model for a Dust 2 match. A Confusion Matrix provides a detailed description of the model’s performance by providing a table showing the relationship between actual labels and the predictions.

Based on the confusion matrix, we can conclude that the model’s predictions are most flawed at the tactic `t_setup_a` (A Setup). Here the model predicts various different tactics wrongly. Some of these tactics, including `t_execute_a_long` or `t_execute_a_short`, are fairly close to being an A Setup based on the features measured. However, there are several tactics that are not even closely related to A setup. `t_execute_mid_to_b` and `t_execute_b` are both positionally very different from an A setup, but the model still predicted them over the actual tactic. Overall, the confusion between `t_setup_a` and execute tactics is somewhat expected, as setup phases often transition into execution phases, creating ambiguous boundaries between features.

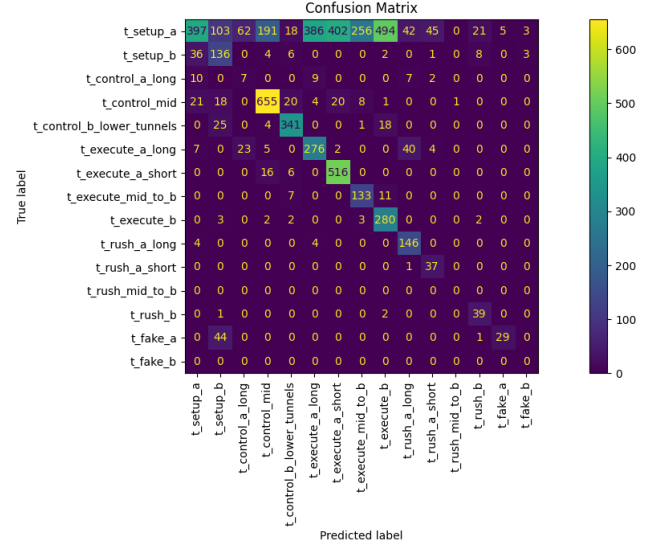


Fig. 2. Confusion matrix for the model’s predictions. The rows represent the actual tactics, while the columns represent the predicted tactics.

5.3 ROC Curve

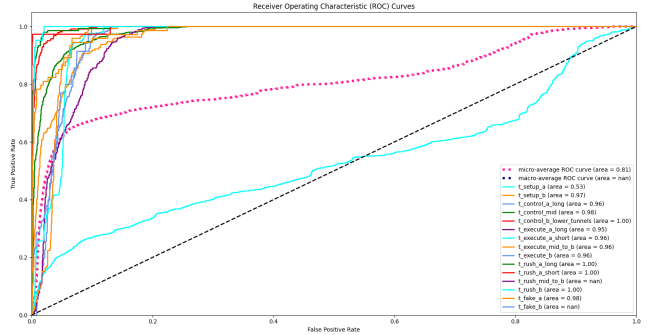


Fig. 3. ROC curve for the model’s predictions. The X axis represents the False Positive rate, while the Y axis represents the True Positive rate.

The Receiver Operating Characteristic (ROC) curve is a widely used tool for evaluating the performance of classification models [7]. The ROC curve plots the true positive rate against the false positive rate for each class at different thresholds, providing insight into the model’s ability to distinguish between tactics. The area under the ROC curve (AUC) is a summary metric that reflects the probability that the model will rank a randomly chosen positive instance higher than a randomly chosen negative one. The closer the value is to 1.0, the better the performance. The dotted black line represents the AUC of a random classifier. Everything to the top-left of it performs better, and everything closer to the bottom-right corner performs worse.

Figure 3 shows the ROC curves for each tactic class predicted by the model. Some tactic classes, such as `t_setup_b`, `t_control_a_long`, and `t_rush_a_long`, achieve high AUC values (0.96–1.00),

indicating that the model distinguishes these tactics well. However, other tactics, such as `t_setup_a`, have a lower AUC (0.53), suggesting that the model struggles to differentiate it. This can be due to overlapping features or insufficient training data for the tactic. The micro-average ROC curve, which summarizes overall model performance across all classes, has an AUC of 0.81, showing decent classification ability on average.

6 DISCUSSION

The results of the research show how effectively GNNs can predict Counter-Strike tactics based on in-game data, achieving results comparable with state-of-the-art approaches. This success highlights the potential improvement a model, like the one proposed in this research, brings to the esports scene.

6.1 Contextualizing results

The results of this research demonstrate that a two-layered Graph Convolutional Network (GCN) using all available node features (position, health, armor, and utility) achieves the highest accuracy and F1-score for predicting Counter-Strike tactics. Specifically, the model reached an accuracy of 81.17% and an F1-score of 0.6945 on the test set. Notably, the inclusion of utility features significantly improved both recall and precision, indicating that utility usage is a strong indicator of tactical intent in Counter-Strike. The confusion matrix (see Figure 2) revealed that the model performs well on most tactical classes but struggles with ambiguous transitions, such as distinguishing between setup and execute phases.

The performance of predictive models in sports and esports analytics is often evaluated using the Area Under the ROC Curve (AUC), a reliable metric for assessing classification models, especially when dealing with imbalanced datasets or multiple classes. In this work, the micro-average AUC across all tactical classes was 0.81 (see Figure 3), indicating that the model can reliably distinguish between most tactics, though it struggles with ambiguous or overlapping classes.

This result is in line with, or even exceeds, the performance reported in similar studies using graph-based or traditional machine learning approaches. For instance, Szmida et al. achieved an AUC of 0.8549 for evaluating player actions in Counter-Strike using Temporal Heterogeneous Graph Neural Networks, while Xenopoulos et al. reported an AUC of 0.8753 for outcome prediction in American football and esports. The findings suggest that the proposed GNN architecture is well-suited for such complex, multi-class problems.

6.2 Implications and takeaways

The strong performance of the model has several important implications for both the esports industry and the broader field of sports analytics. First, the ability to accurately predict tactics from partial in-game data opens new opportunities for automated coaching tools and real-time tactical feedback. Coaches and analysts can leverage such models to identify patterns, evaluate team performance, and develop targeted training based on objective data rather than intuition alone.

The findings also suggest that utility usage (such as grenade throws and flashbangs) plays a crucial role in tactic recognition,

which could inform future research and practical applications. For example, teams may focus on optimizing utility usage to maximize tactical effectiveness, while automated systems could use these insights to label tactical phases in large datasets or to simulate “what-if” scenarios for strategy development.

From a research standpoint, these results add to the increasing amount of data demonstrating the efficiency of GNNs for analytics in esports and sports. The outcomes are consistent with recent research showing that GNNs can statistically outperform conventional techniques in modeling intricate spatial and relational patterns [6]. In particular, the GCN architecture’s performance in this situation indicates that, as long as the graph representation accurately describes the relational structure, even comparatively basic GNNs can perform exceptionally well on tasks requiring the prediction of team-level strategies. This insight is particularly relevant for researchers working on predictive modeling in fields where local and global interactions are critical.

Furthermore, the findings emphasize how crucial feature representation and selection are. Future research should focus on identifying the features that best capture tactical intent in team-based games, as the inclusion of utility features, such as grenade usage, was found to be essential for model performance. This result is in line with recent research in sports analytics, which has demonstrated that incorporating sophisticated features enhances model interpretability and accuracy [6].

6.3 Limitation of research

Several limitations should be considered regarding the outcome of the research. Firstly, the accuracy and consistency of the labeled data used for training and testing the model is dependent on human annotators. As a result, the labels can be affected by subjectivity and inconsistencies. This introduces a possibility of biases and noise into the training and testing data, which can affect the model’s performance.

Furthermore, while the GCN architecture yielded strong results, there is a chance that other, more complex architectures, such as Deep Graph Convolutional Networks, could be more accurate when dealing with larger and more complex datasets. The optimal number of layers and architecture style may depend on the complexity of the training data. The model was also developed and trained exclusively on Counter-Strike matches played on the Dust 2 map, meaning that the performance of the model on other maps remains uncertain. Every map has different layouts and tactical patterns; therefore, further research is needed in order to explore the model’s capabilities on a broader scale.

Finally, the research did not explore the impact of different feature sets or alternative graph representations. For example, incorporating additional features, such as player roles or equipment loadouts, could further improve model performance. Similarly, modeling graphs with different edge definitions or incorporating temporal dynamics could provide new insights into tactical behavior.

6.4 Future research

This research offers a product that can later be utilized in several different ways depending on the needs of the user.

Firstly, the model, being able to predict the tactics in a round, can be used to automatically label Counter-Strike datasets. This can be especially useful for data-driven tactical approaches of teams who want to reflect on their performances on certain maps throughout a specific period of time. It is possible, then, to draw conclusions about which tactic worked statistically better than others.

Furthermore, the model can be adjusted so that it performs real-time predictions on games. This, extended with the model having knowledge about past performances of the team, can work as a real-time AI assistant for Counter-Strike matches. This product would give advice on whether the team should continue with their current tactic or switch to another one, based on historical data on the team's performance while carrying out that tactic.

Lastly, it is possible to implement the model into the game itself, where a team full of NPCs (Non-Player Characters) could coordinate tactics. They could execute even higher-level tactics because of the knowledge of these tactics that the model has gained throughout training.

A APPENDIX A: USE OF AI

During this research, Szabolcs Csapo used Perplexity AI in order to improve the coherence of the text in the paper and to generate example code. After using this tool, the content was reviewed and edited as needed, and Szabolcs Csapo takes full responsibility for the content of the work.

REFERENCES

- [1] Clement, J. (2025, April 7). eSports market – statistics & facts. Statista. <https://www.statista.com/topics/3121/esports-market/>
- [2] Wang, Z., Veličković, P., Hennes, D., et al. (2024). TacticAI: an AI assistant for football tactics. *Nature Communications*, 15, 1906. <https://doi.org/10.1038/s41467-024-45965-x>
- [3] Müller, O., Caron, M., Döring, M., Heuwinkel, T., & Baumeister, J. (2022). PIVOT: A parsimonious end-to-end learning framework for valuing player actions in handball using tracking data. In U. Brefeld, J. Davis, J. Van Haaren, & A. Zimmermann (Eds.), *Machine Learning and Data Mining for Sports Analytics* (pp. 129–145). Springer. https://doi.org/10.1007/978-3-031-02044-5_10
- [4] Mora-Cantalops, M., & Sicilia, M.-Á. (2019). Team efficiency and network structure: The case of professional League of Legends. *Social Networks*, 58, 105–115. <https://doi.org/10.1016/j.socnet.2019.03.004>
- [5] Xenopoulos, P., & Silva, C. (2022). ESTA: an Esports Trajectory and Action Dataset. arXiv.org. <https://doi.org/10.48550/arXiv.2209.09861>
- [6] Xenopoulos, P., & Silva, C. (2021). Graph neural networks to predict sports outcomes. In *2021 IEEE International Conference on Big Data (Big Data)* (pp. 1757–1763). <https://doi.org/10.1109/BigData52589.2021.9671833>
- [7] Sanchez-Lengeling, B., Reif, E., Pearce, A., & Wiltchko, A. (2021). A gentle introduction to graph neural networks. *Distill*, 6(8). <https://doi.org/10.23915/distill.00033>
- [8] Szmida, P. P., & Toka, L. (2019). Evaluating player actions in professional Counter-Strike using temporal heterogeneous graph neural networks. In *Proceedings of the MIT Sloan Sports Analytics Conference*. https://www.researchgate.net/publication/391055455_Evaluating_Player_Actions_in_Professional_Counter_Strike_using_Temporal_Heterogeneous_Graph_Neural_Networks
- [9] Loshchilov, I., & Hutter, F. (2017, November 14). Decoupled weight decay regularization. arXiv.org. <https://arxiv.org/abs/1711.05101>